

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 12

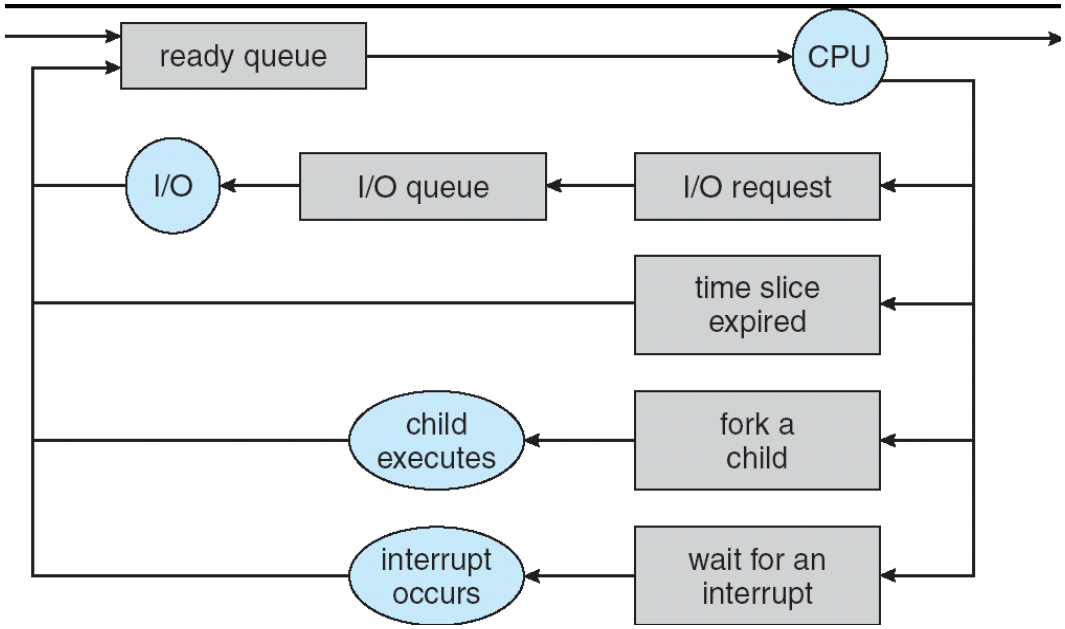
Scheduling:
fair sharing, lottery scheduling, EEVDF

Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

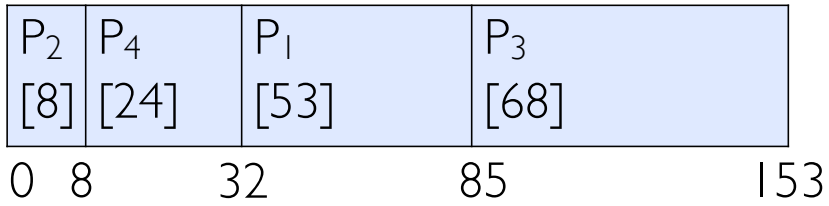
Recall: scheduling



- Question: how is the OS to decide which of several tasks to take off a queue?
- Scheduling: deciding which threads are given access to resources from moment to moment
  - Often, we think in terms of CPU time, but could also think about access to resources like network bandwidth or disk access

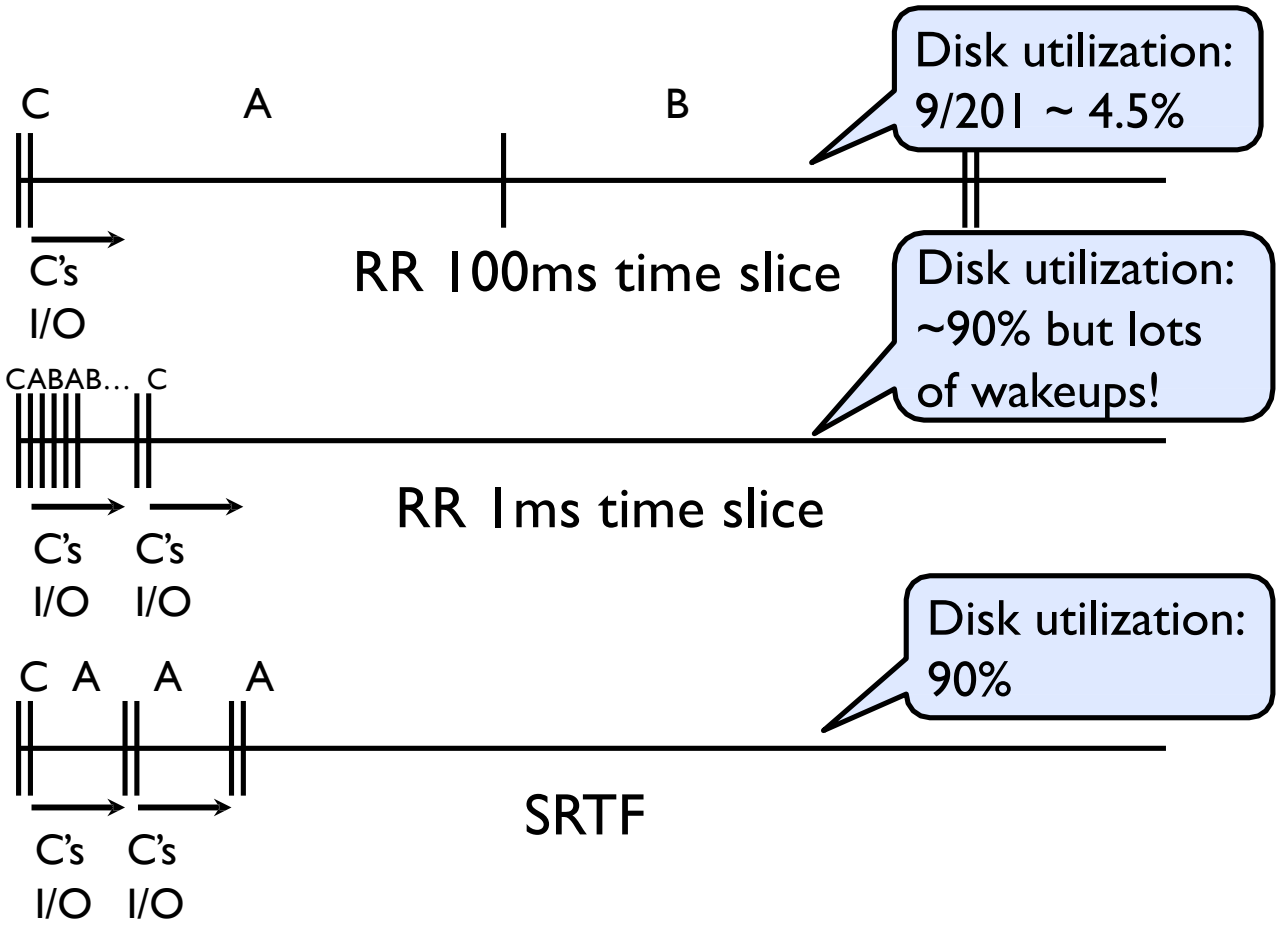
Recall: FCFS/FIFO vs. RR

Best FCFS:



	Quantum	P_1	P_2	P_3	P_4	Average
Wait Time	Best FCFS	32	0	85	8	31¼
	Q = 1	84	22	85	57	62
	Q = 5	82	20	85	58	61¼
	Q = 8	80	8	85	56	57¼
	Q = 10	82	10	85	68	61¼
	Q = 20	72	20	85	88	66¼
	Worst FCFS	68	145	0	121	83½
Completion Time	Best FCFS	85	8	153	32	69½
	Q = 1	137	30	153	81	100½
	Q = 5	135	28	153	82	99½
	Q = 8	133	16	153	80	95½
	Q = 10	135	18	153	92	99½
	Q = 20	125	28	153	112	104½
	Worst FCFS	121	153	68	145	121¾

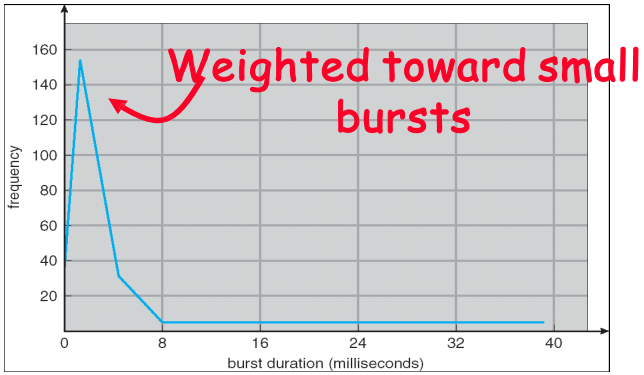
Recall: SRTF


C
A
B
Disk utilization:
9/201 ~ 4.5%
C's
I/O
RR 100ms time slice
Disk utilization:
~90% but lots
of wakeups!
CABAB… C
C's
I/O
C's
I/O
RR 1ms time slice
C A A A
Disk utilization:
90%
C's
I/O
C's
I/O
SRTF

Multi-level feedback scheduling

How to handle simultaneous mix of different types of apps?

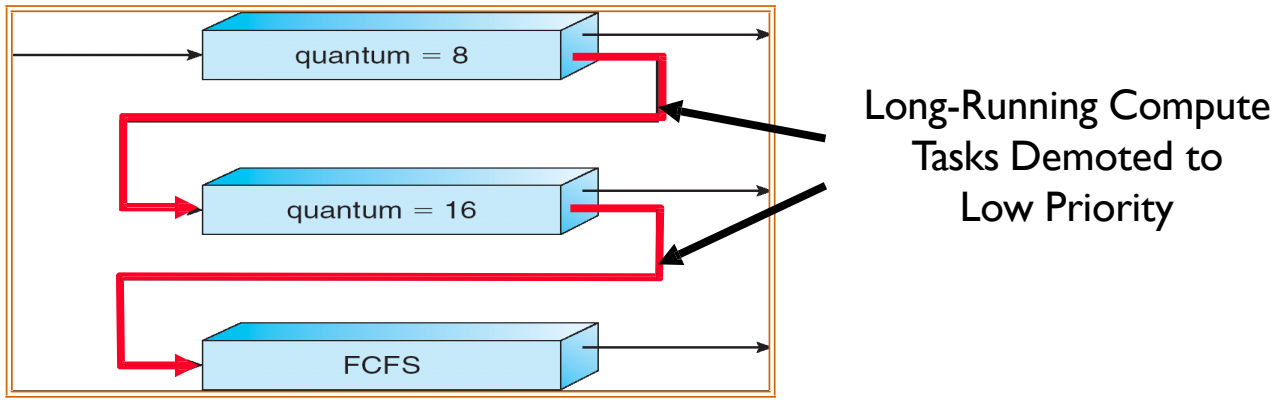
- Consider mix of interactive and high throughput apps:
  - How to best schedule them?
  - How to recognize one from the other?
    - Do you trust app to say that it is “interactive”?
  - Should you schedule the set of apps identically on servers, workstations, pads, and cellphones?
- For instance, is Burst Time (observed) useful to decide which application gets CPU time?
  - Short Bursts ⇒ Interactivity ⇒ High Priority?



How to handle simultaneous mix of different types of apps?

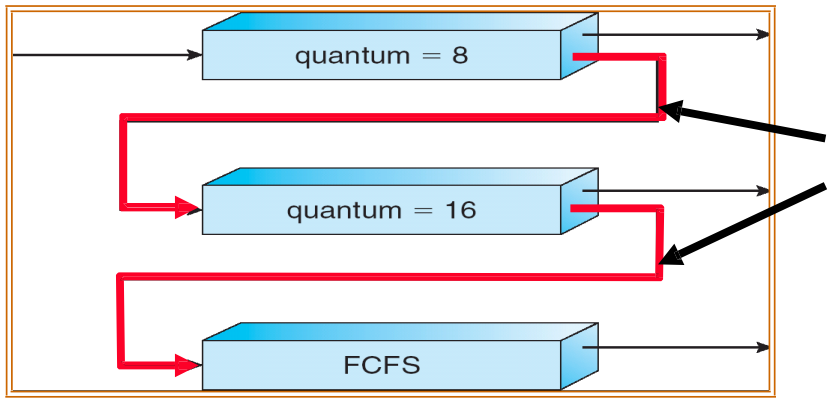
- Assumptions encoded into many schedulers:
  - Apps that sleep a lot and have short bursts must be interactive apps – they should get high priority
  - Apps that compute a lot should get low(er?) priority, since they won't notice intermittent bursts from interactive apps
- Hard to characterize apps:
  - What about apps that sleep for a long time, but then compute for a long time?
  - Or, what about apps that must run under all circumstances (say periodically)

Multi-level feedback scheduling



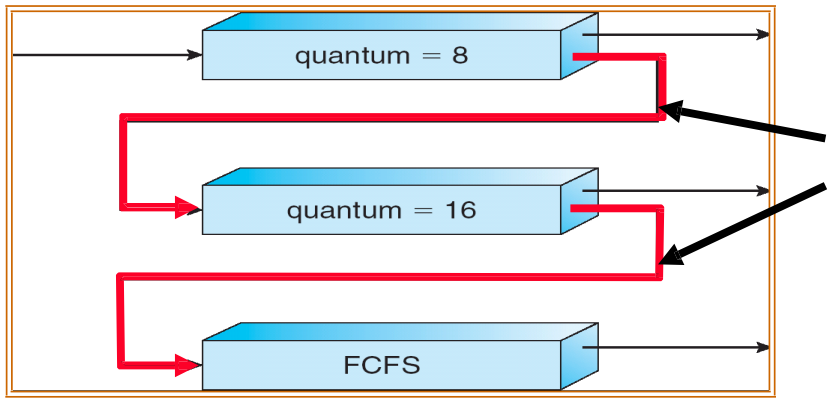
- Another method for exploiting past behavior
  - Multiple queues, each with different priority
    - Higher priority queues often considered “foreground” tasks
  - Each queue has its own scheduling algorithm
    - e.g. foreground – RR, background – FCFS
    - Sometimes multiple RR priorities with quantum increasing exponentially (highest:1ms, next: 2ms, next: 4ms, etc)

Multi-level feedback scheduling



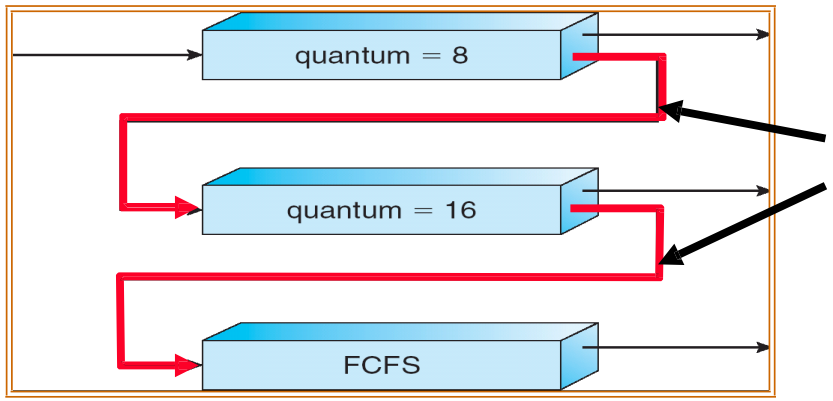
- Adjust each job's priority as follows (details vary)
  - Job starts in highest priority queue
  - If timeout expires, drop one level
  - If timeout doesn't expire, push up one level (or to top)

Multi-level feedback scheduling



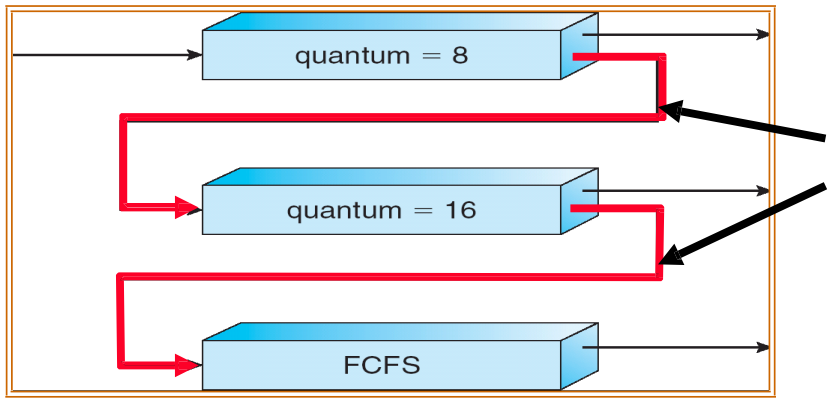
- Result approximates SRTF:
  - CPU bound jobs drop like a rock
  - Short-running I/O bound jobs stay near top

Multi-level feedback scheduling



- Scheduling must be done between the queues
  - Fixed priority scheduling:
    - serve all from highest priority, then next priority, etc.
  - Time slice:
    - each queue gets a certain amount of CPU time
    - e.g., 70% to highest, 20% next, 10% lowest

Multi-level feedback scheduling



- Countermeasure: user action that can foil intent of the OS designers
  - For multilevel feedback, put in a bunch of meaningless I/O to keep job's priority high
  - Of course, if everyone did this, wouldn't work!
- Example of Othello program:
  - Playing against competitor, so key was to do computing at higher priority than the competitors
    - Put in printf's, ran much faster!

Case study: Linux O(1) scheduler



- Priority-based scheduler: 140 priorities
  - 40 for “user tasks” (set by “nice”), 100 for “Realtime/Kernel”
  - Lower priority value ⇒ higher priority (for realtime values)
  - Highest priority value ⇒ Lower priority (for nice values)
  - All algorithms O(1)
    - Timeslices/priorities/interactivity credits all computed when job finishes time slice
    - 140-bit mask indicates presence or absence of job at given priority level

Case study: Linux O(1) scheduler



- Two separate priority queues: “active” and “expired”
  - All tasks in the active queue use up their timeslices and get placed on the expired queue, after which queues swapped
- Timeslice depends on priority – linearly mapped onto timeslice range
  - Like a multi-level queue (one queue per priority) with different timeslice at each level
  - Execution split into “Timeslice Granularity” chunks – round robin through priority

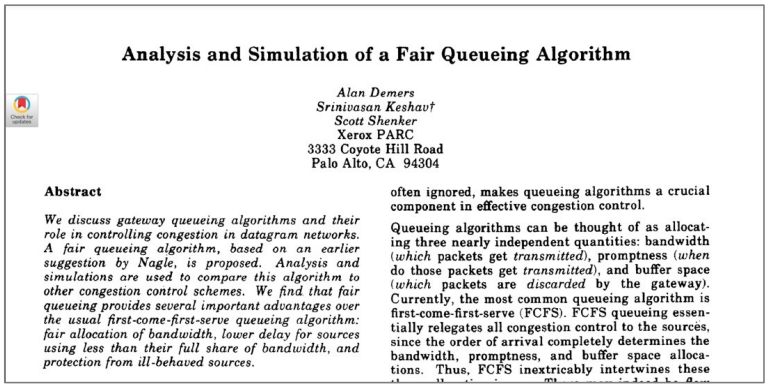
Case study: Linux O(1) scheduler

- Heuristics
  - User-task priority adjusted ±5 based on heuristics
    - P → sleep_avg = sleep_time – run_time
    - Higher sleep_avg ⇒ more I/O bound the task, more reward (and vice versa)
  - Interactive Credit (IC)
    - Earned when a task sleeps for a “long” time
    - Spend when a task runs for a “long” time
    - IC is used to provide hysteresis to avoid changing interactivity for temporary changes in behavior
  - However, “interactive tasks” get special dispensation
    - To try to maintain interactivity
    - Placed back into active queue, unless some other task has been starved for too long…

Fair sharing

Scheduling fairness

- Two identical processes should get
  - the “same” service, experience the “same” performance
- Work-conserving scheduler: does not leave the CPU idle when there is work to do
- Note: fairness avoids starvation
- Example: round-robin



Analysis and Simulation of a Fair Queueing Algorithm

Alan Demers
Srinivasan Keshav†
Scott Shenker
Xerox PARC
3333 Coyote Hill Road
Palo Alto, CA 94304

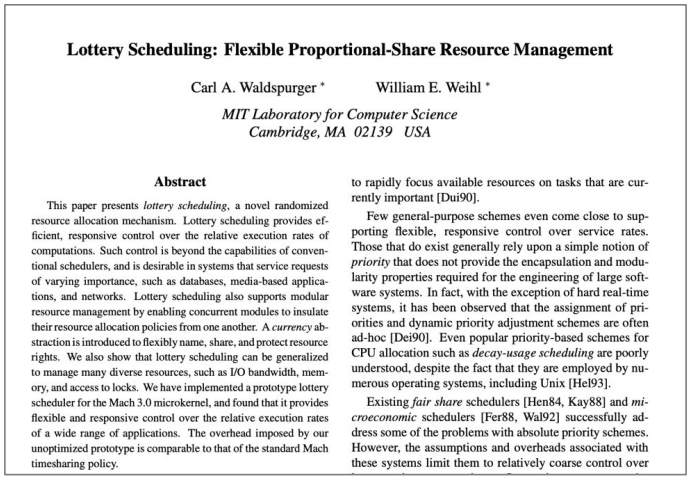
Abstract

We discuss gateway queueing algorithms and their role in controlling congestion in datagram networks. A fair queueing algorithm, based on an earlier suggestion by Nagle, is proposed. Analysis and simulations are used to compare this algorithm to other congestion control schemes. We find that fair queueing provides several important advantages over the usual first-come-first-serve queueing algorithm: fair allocation of bandwidth, lower delay for sources using less than their full share of bandwidth, and protection from ill-behaved sources.

often ignored, makes queueing algorithms a crucial component in effective congestion control.

Queueing algorithms can be thought of as allocating three nearly independent quantities: bandwidth (*which* packets get *transmitted*), promptness (*when* do those packets get *transmitted*), and buffer space (*which* packets are *discarded* by the gateway). Currently, the most common queueing algorithm is first-come-first-serve (FCFS). FCFS queueing essentially relegates all congestion control to the sources, since the order of arrival completely determines the bandwidth, promptness, and buffer space allocations. Thus, FCFS inextricably intertwines these

SIGCOMM ‘89



Lottery Scheduling: Flexible Proportional-Share Resource Management

Carl A. Waldspurger * William E. Weihl *

MIT Laboratory for Computer Science
Cambridge, MA 02139 USA

Abstract

This paper presents *lottery scheduling*, a novel randomized resource allocation mechanism. Lottery scheduling provides efficient, responsive control over the relative execution rates of computations. Such control is beyond the capabilities of conventional schedulers, and is desirable in systems that service requests of varying importance, such as databases, media-based applications, and networks. Lottery scheduling also supports modular resource management by enabling concurrent modules to insulate their resource allocation policies from one another. A *currency* abstraction is introduced to flexibly name, share, and protect resource rights. We also show that lottery scheduling can be generalized to manage many diverse resources, such as I/O bandwidth, memory, and access to locks. We have implemented a prototype lottery scheduler for the Mach 3.0 microkernel, and found that it provides flexible and responsive control over the relative execution rates of a wide range of applications. The overhead imposed by our unoptimized prototype is comparable to that of the standard Mach timesharing policy.

to rapidly focus available resources on tasks that are currently important [Dui90].

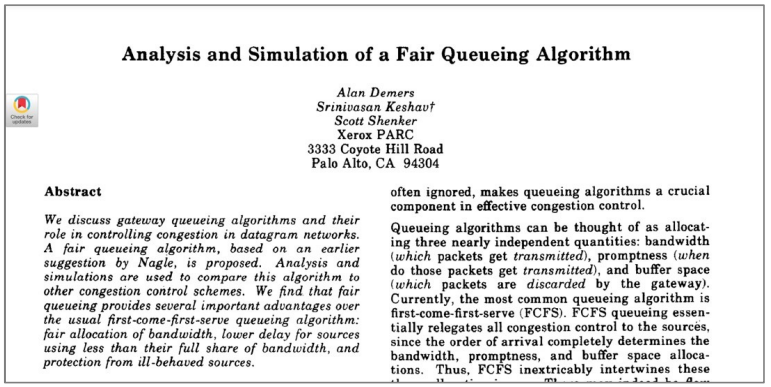
Few general-purpose schemes even come close to supporting flexible, responsive control over service rates. Those that do exist generally rely upon a simple notion of *priority* that does not provide the encapsulation and modularity properties required for the engineering of large software systems. In fact, with the exception of hard real-time systems, it has been observed that the assignment of priorities and dynamic priority adjustment schemes are often ad-hoc [Dei90]. Even popular priority-based schemes for CPU allocation such as *decay-usage scheduling* are poorly understood, despite the fact that they are employed by numerous operating systems, including Unix [Hel93].

Existing *fair share* schedulers [Hen84, Kay88] and *microeconomic* schedulers [Fer88, Wal92] successfully address some of the problems with absolute priority schemes. However, the assumptions and overheads associated with these systems limit them to relatively coarse control over

OSDI ‘94

Scheduling fairness

- Next:
  - Fair sharing / Fair queueing / Max-min fairness)
  - Originated in networking
  - Later applied to CPU scheduling and other resources (lottery scheduling)



Analysis and Simulation of a Fair Queueing Algorithm

Alan Demers
Srinivasan Keshav†
Scott Shenker
Xerox PARC
3333 Coyote Hill Road
Palo Alto, CA 94304

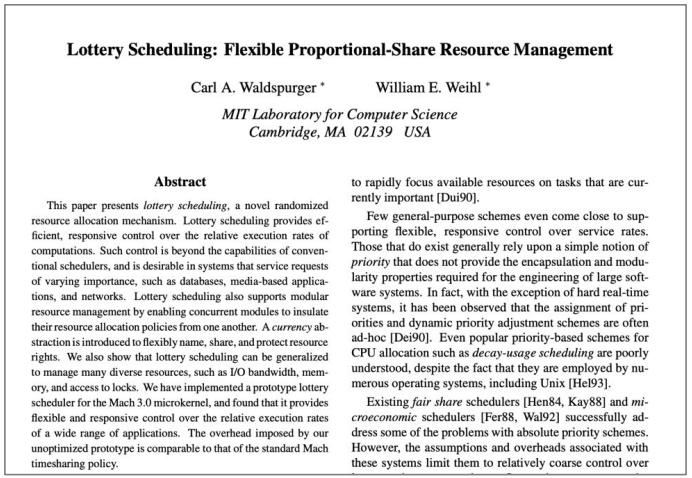
Abstract

We discuss gateway queueing algorithms and their role in controlling congestion in datagram networks. A fair queueing algorithm, based on an earlier suggestion by Nagle, is proposed. Analysis and simulations are used to compare this algorithm to other congestion control schemes. We find that fair queueing provides several important advantages over the usual first-come-first-serve queueing algorithm: fair allocation of bandwidth, lower delay for sources using less than their full share of bandwidth, and protection from ill-behaved sources.

often ignored, makes queueing algorithms a crucial component in effective congestion control.

Queueing algorithms can be thought of as allocating three nearly independent quantities: bandwidth (*which* packets get *transmitted*), promptness (*when* do those packets get *transmitted*), and buffer space (*which* packets are *discarded* by the gateway). Currently, the most common queueing algorithm is first-come-first-serve (FCFS). FCFS queueing essentially relegates all congestion control to the sources, since the order of arrival completely determines the bandwidth, promptness, and buffer space allocations. Thus, FCFS inextricably intertwines these

SIGCOMM '89



Lottery Scheduling: Flexible Proportional-Share Resource Management

Carl A. Waldspurger * William E. Weihl *

MIT Laboratory for Computer Science
Cambridge, MA 02139 USA

Abstract

This paper presents *lottery scheduling*, a novel randomized resource allocation mechanism. Lottery scheduling provides efficient, responsive control over the relative execution rates of computations. Such control is beyond the capabilities of conventional schedulers, and is desirable in systems that service requests of varying importance, such as databases, media-based applications, and networks. Lottery scheduling also supports modular resource management by enabling concurrent modules to insulate their resource allocation policies from one another. A *currency* abstraction is introduced to flexibly name, share, and protect resource rights. We also show that lottery scheduling can be generalized to manage many diverse resources, such as I/O bandwidth, memory, and access to locks. We have implemented a prototype lottery scheduler for the Mach 3.0 microkernel, and found that it provides flexible and responsive control over the relative execution rates of a wide range of applications. The overhead imposed by our unoptimized prototype is comparable to that of the standard Mach timesharing policy.

to rapidly focus available resources on tasks that are currently important [Dui90].

Few general-purpose schemes even come close to supporting flexible, responsive control over service rates. Those that do exist generally rely upon a simple notion of *priority* that does not provide the encapsulation and modularity properties required for the engineering of large software systems. In fact, with the exception of hard real-time systems, it has been observed that the assignment of priorities and dynamic priority adjustment schemes are often ad-hoc [Dei90]. Even popular priority-based schemes for CPU allocation such as *decay-usage scheduling* are poorly understood, despite the fact that they are employed by numerous operating systems, including Unix [Hel93].

Existing *fair share* schedulers [Hen84, Kay88] and *microeconomic* schedulers [Fer88, Wal92] successfully address some of the problems with absolute priority schemes. However, the assumptions and overheads associated with these systems limit them to relatively coarse control over

OSDI '94

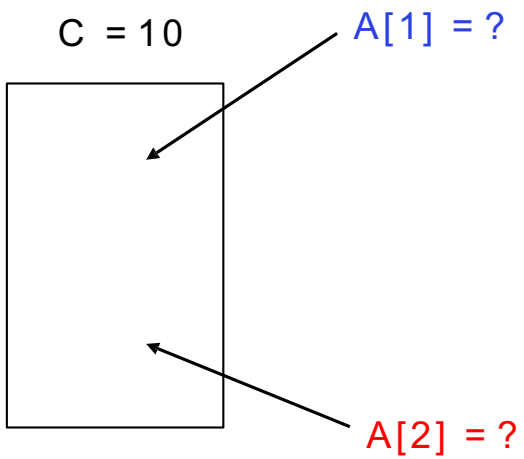
Fair allocation: example 1

Resource with capacity, C = 10

Two jobs with demands D[1] = 8, D[2] = 6

How much should each job get?

- A[1] = ?
- A[2] = ?



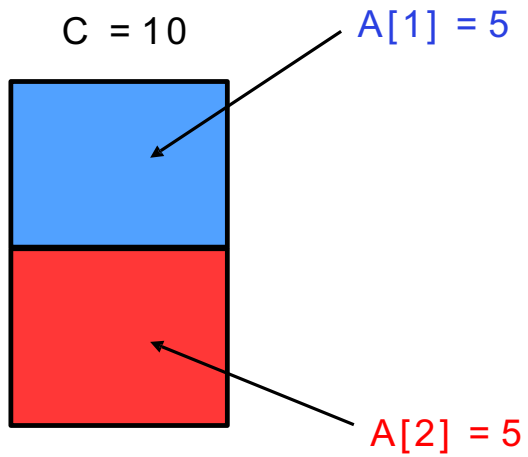
Fair allocation: example 1

Resource with capacity, C = 10

Two jobs with demands D[1] = 8, D[2] = 6

How much should each job get?

- A[1] = 5
- A[2] = 5



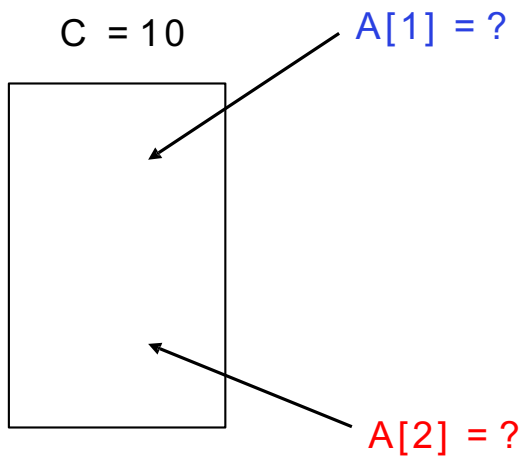
Fair allocation: example 2

Resource with capacity, C = 10

Two jobs with demands D[1] = 8, D[2] = **4**

How much should each job get?

- A[1] = ?
- A[2] = ?



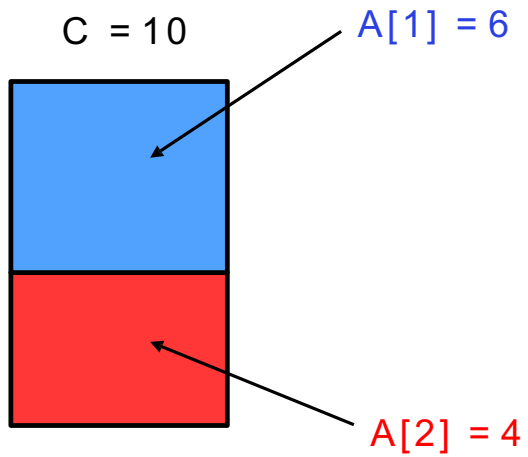
Fair allocation: example 2

Resource with capacity, C = 10

Two jobs with demands D[1] = 8, D[2] = 4

How much should each job get?

- A[1] = 6
- A[2] = 4



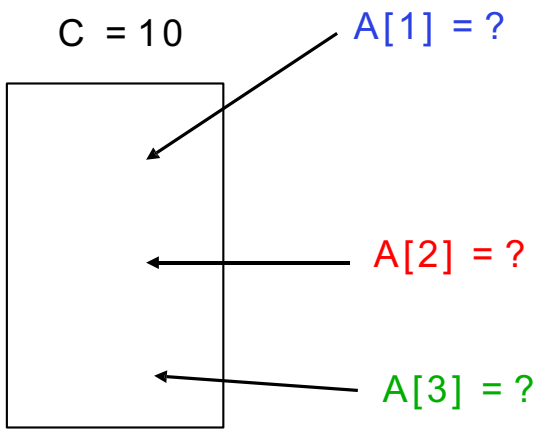
Fair allocation: example 3

Resource with capacity, C = 10

Three jobs with demands D[1] = 8, D[2] = 6, **D[3] = 2**

How much should each job get?

- A[1] = ?
- A[2] = ?
- A[3] = ?



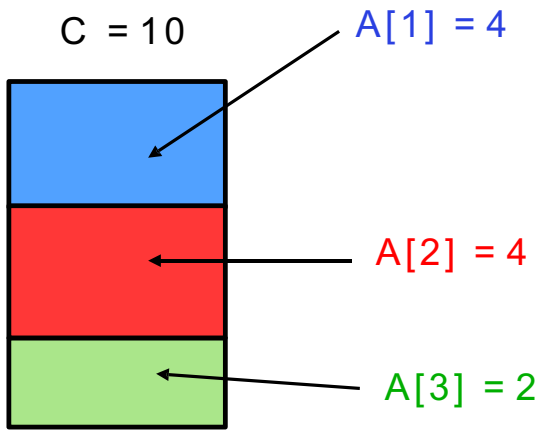
Fair allocation: example 3

Resource with capacity, C = 10

Three jobs with demands D[1] = 8, D[2] = 6, D[3] = 2

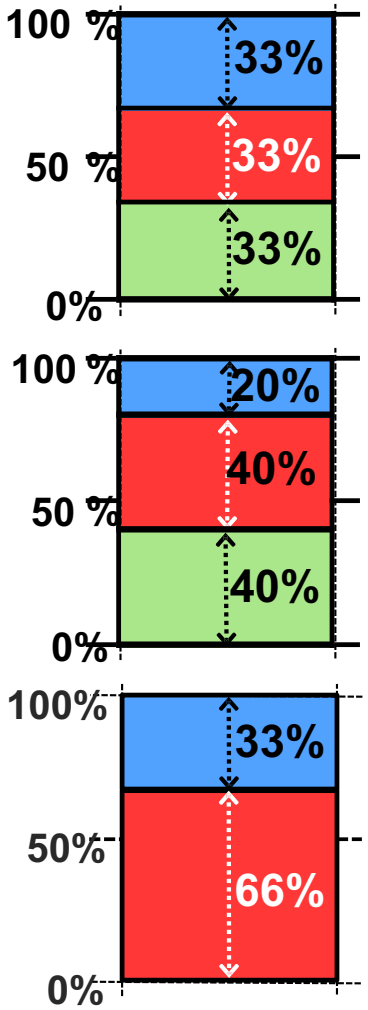
How much should each job get?

- A[1] = ?
- A[2] = ?
- A[3] = ?



Single resource: fair sharing

- *n* jobs want to share a resource (e.g. CPU)
  - Solution: give each *1/n* of the shared resource
  - Assume *n* = 3
- Generalized by **max-min fairness**
  - Handles if a job wants less than its fair share
  - E.g. job 1 wants no more than 20%
- Generalized by **weighted max-min fairness**
  - Give weights to jobs according to importance
  - Job 1 gets weight 1, job 2 weight 2



Why max-min fairness?

- *Weighted Fair Sharing / Proportional Shares*
  - User 1 gets weight 2, user 2 weight 1
- *Priorities*
  - Give user 1 weight 1000, user 2 weight 1
- *Reservations*
  - Ensure user 1 gets 10% of a resource
  - Give user 1 weight 10, sum weights ≤ 100
- *Deadline-based scheduling*
  - Given a user job's demand and deadline, compute user's reservation/weight
- *Isolation*
  - Users cannot affect others beyond their share

Widely used

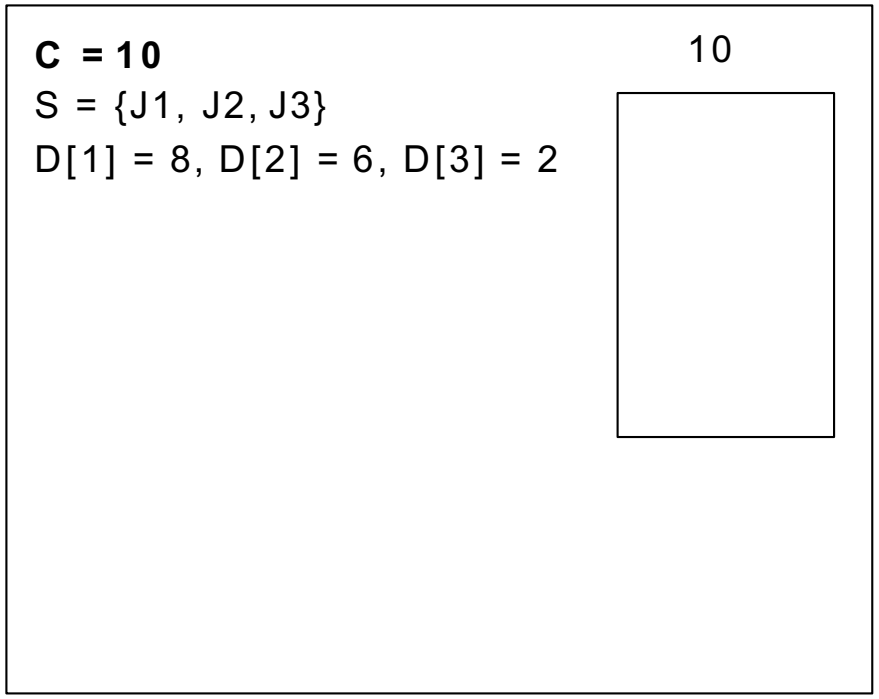
- ***OS***: proportional sharing, lottery, Linux's CFS, …
- ***Networking***: wfq, wf2q, sfq, drr, csfq, ...
- ***Datacenters***: Hadoop's fair sched, Dominant Resource Fairness (DRF)

Max-min fairness

```
C # resource capacity
S = {J1, J2, …, Jn} # set of n jobs
D[n] # demand vector; D[i]: demand of job i
A[n] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every Ji in S:
    if D[i] < f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {Ji}
  if b:
    for every Ji in S:
      A[i] = f
      S = S \ {Ji}
```

Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```



Max-min fairness: running example

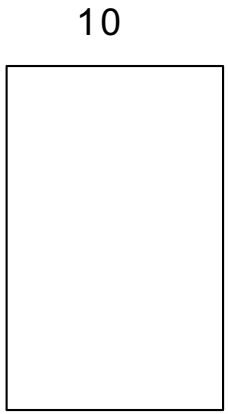
```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```

C = 10

S = {J1, J2, J3}

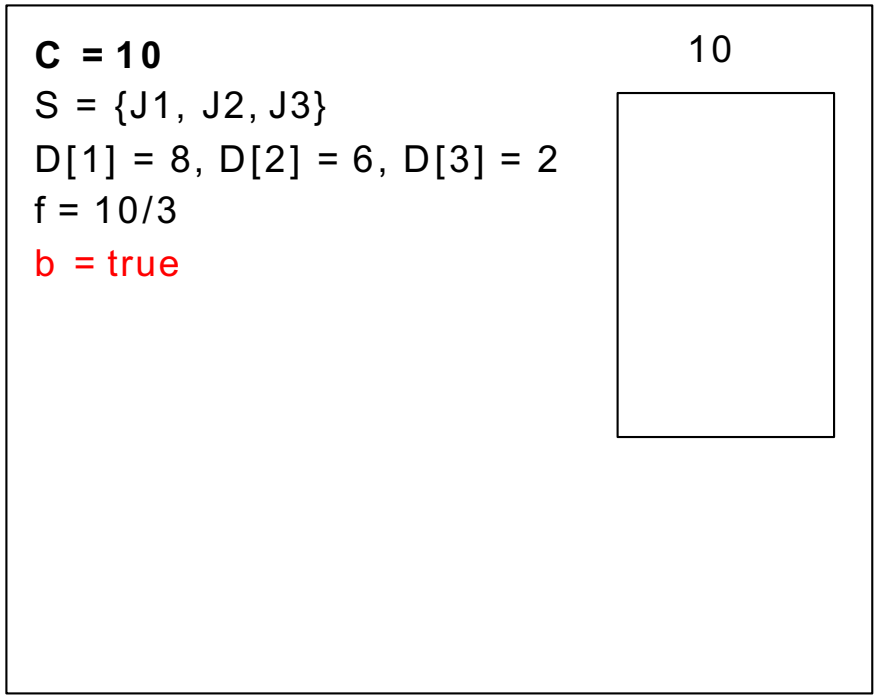
D[1] = 8, D[2] = 6, D[3] = 2

f = 10/3



Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```



Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```

C = 10

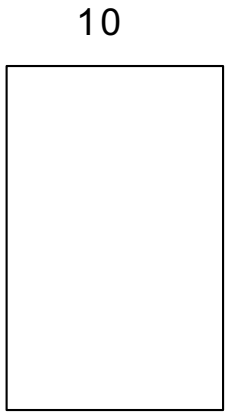
S = {J1, J2, J3}

D[1] = 8, D[2] = 6, D[3] = 2

f = 10/3

b = true

D[3] < 10/3 → A[3] = D[3]



Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```

C = 10 -2 = 8

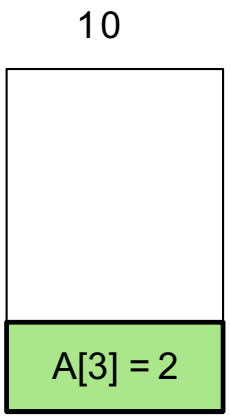
S = {J1, J2}

D[1] = 8, D[2] = 6, D[3] = 2

f = 10/3

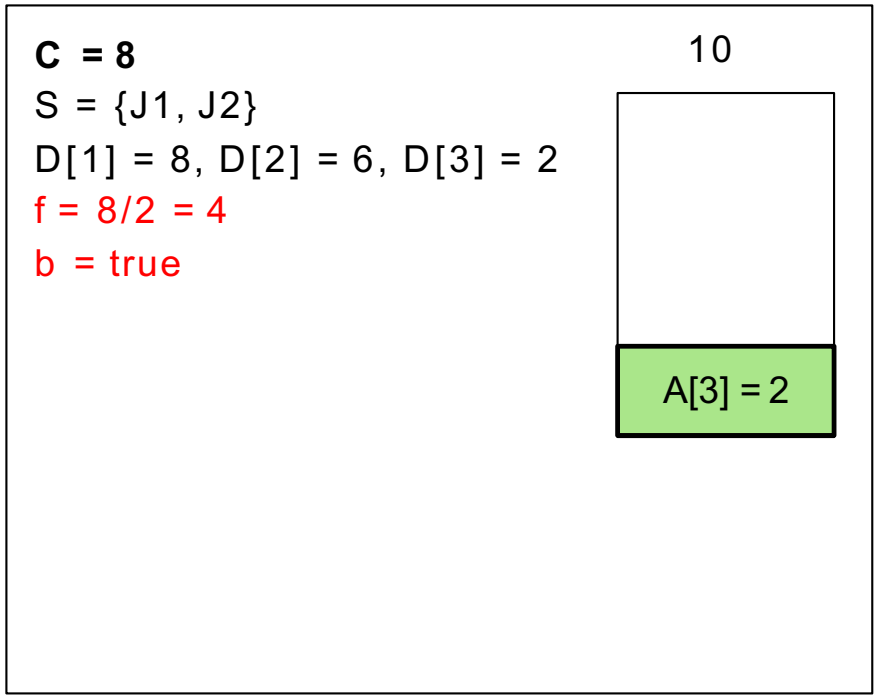
b = false

D[3] < 10/3 → A[3] = D[3]



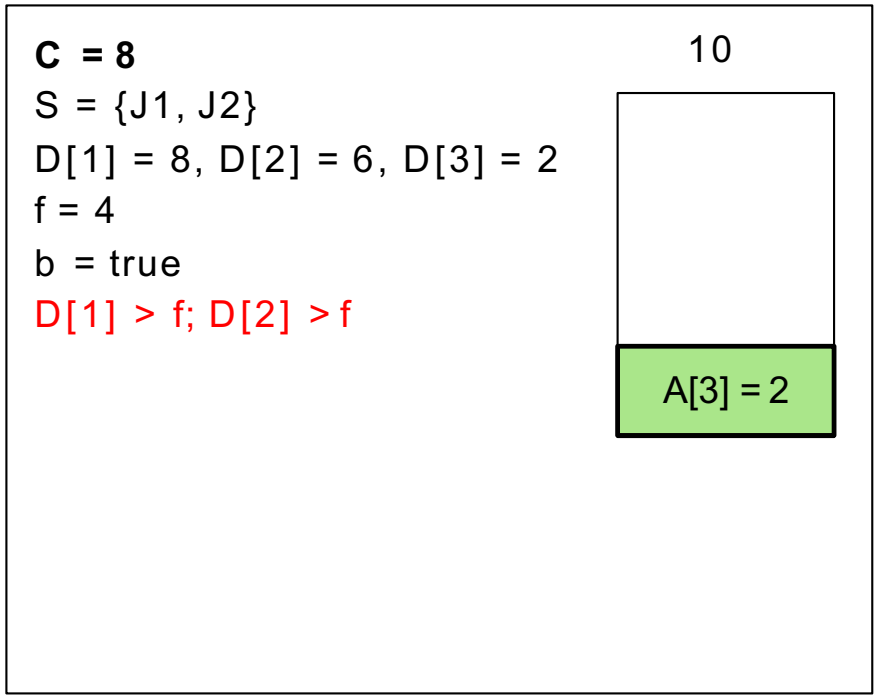
Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```



Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
  if b:
    for every j in S:
      A[i] = f
      S = S \ {j}
```



Max-min fairness: running example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
  f = C/|S| # fair share
  b = true # no job in S has demand < f
  for every j in S:
    if D[i] <= f
      b = false
      A[i] = D[i]
      C = C – D[i]
      S = S \ {j}
if b:
  for every j in S:
    A[i] = f
    S = S \ {j}
```

C = 8

S = {}

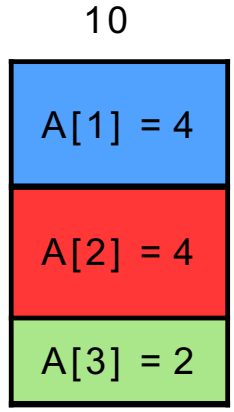
D[1] = 8, D[2] = 6, D[3] = 2

f = 4

b = true

A[1] = 4

A[2] = 4

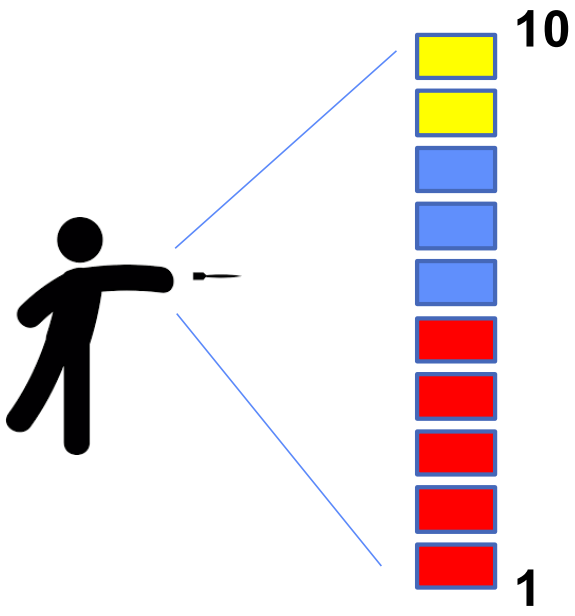


Lottery scheduling

- An approximation of weighted fair sharing
  - Weight → number of tickets
  - Scheduling decision → probabilistic: give a slot to a process proportionally to its weight



Lottery scheduling: simple mechanism



- $N_{ticket} = \Sigma\ \mathbf{N_i}$
- Pick a number d in $1..\ N_{ticket}$ as the random “dart”
- Jobs record their $\mathbf{N_i}$ of allocated tickets
- Order them by $\mathbf{N_i}$
- Select the first j such that $\Sigma\ \mathbf{N_i}$ up to j exceeds d.

Lottery scheduling example (cont.)

- Lottery Scheduling Example
  - Assume short jobs get 10 tickets, long jobs get 1 ticket

# short jobs/ # long jobs	% of CPU each short jobs gets	% of CPU each long jobs gets
1/1	91%	9%
0/2	N/A	50%
2/0	50%	N/A
10/1	9.9%	0.99%
1/10	50%	5%

  - What if too many short jobs to give reasonable response time?
    - If load average is 100, hard to make progress
    - One approach: log some user out

Weighted fair queueing

Fair queueing: max-min fairness originated in networking

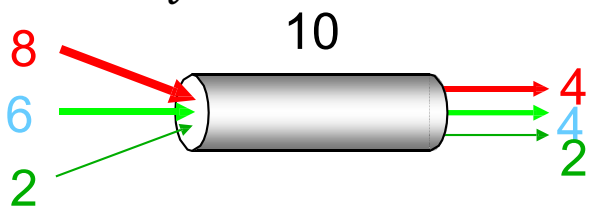
- Fair queueing explained in a fluid flow system: link (resource) allocated in arbitrary small quanta
  - Each flow receives $min(r_i, f)$, where
    - r_i – flow arrival rate
    - f – link fair rate (see next slide)
- Weighted Fair Queueing (WFQ) – associate a weight with each flow [Demers, Keshav & Shenker '89]
  - In networking, a fluid flow system reduces to bit-by-bit round robin
- WFQ in a fluid flow system → Generalized Processor Sharing (GPS) [Parekh & Gallager '92]

Fair rate computation

- If link congested, compute f such that

This is the same value as computed by max-min fair algorithm

$$\sum_i \min(r_i, f) = C$$

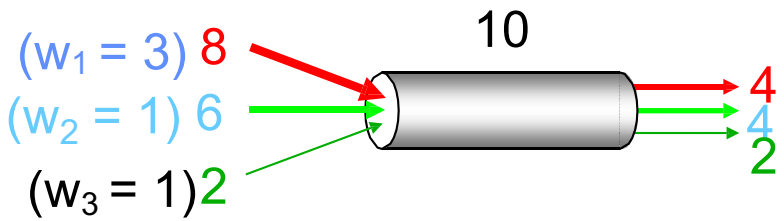


f = 4:
min(8, 4) = 4
min(6, 4) = 4
min(2, 4) = 2

Fair rate computation

- Associate a weight w_i with each flow i
- If link congested, compute f such that

$$\sum_i \min(r_i, f \cdot w_i) = C$$



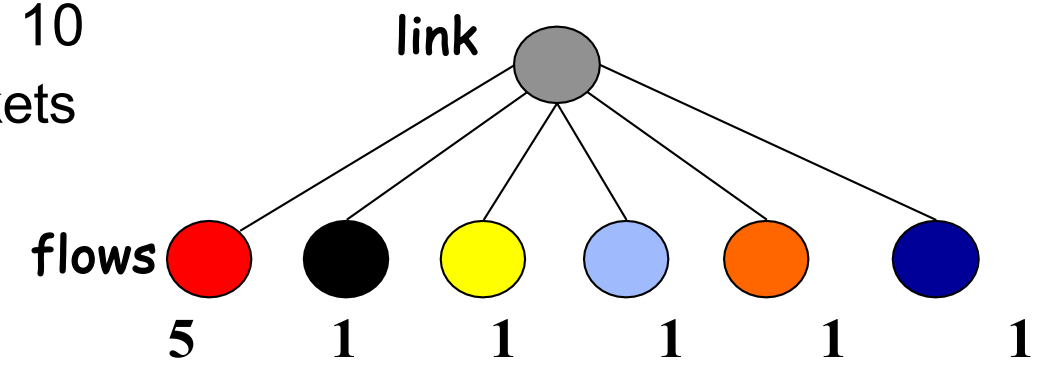
f = 2:
min(8, 2*3) = 6
min(6, 2*1) = 2
min(2, 2*1) = 2

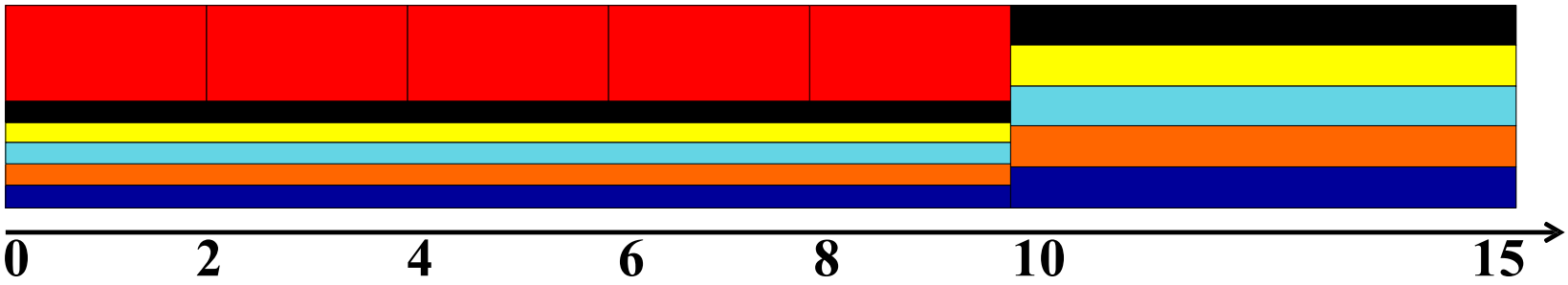
Fluid flow system

- Flows can be served one bit at a time
  - Fluid flow system, also known as Generalized Processor Sharing (GPS) [Parekh and Gallager '93]
- WFQ can be implemented using bit-by-bit weighted round robin in GPS model
  - During each round from each flow that has data to send, send a number of bits equal to the flow's weight

Generalized processor sharing example

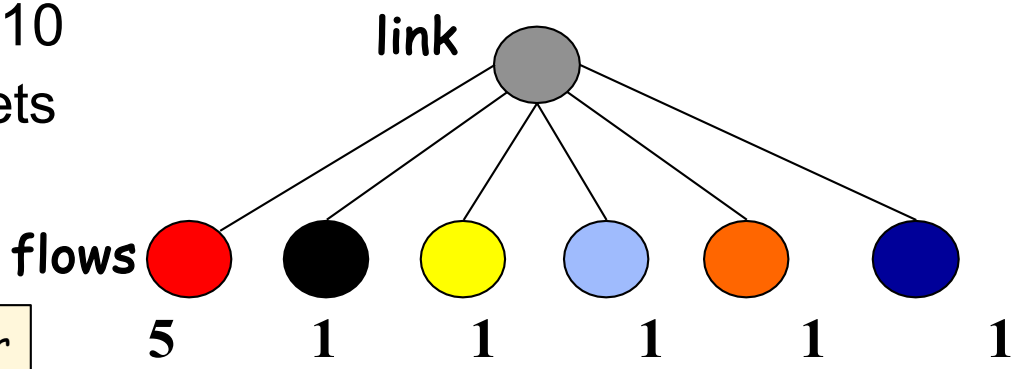
- Red session has packets backlogged between 0 and 10
  - Other sessions have packets continuously backlogged
- Each packet has size 1
- Link capacity is 1

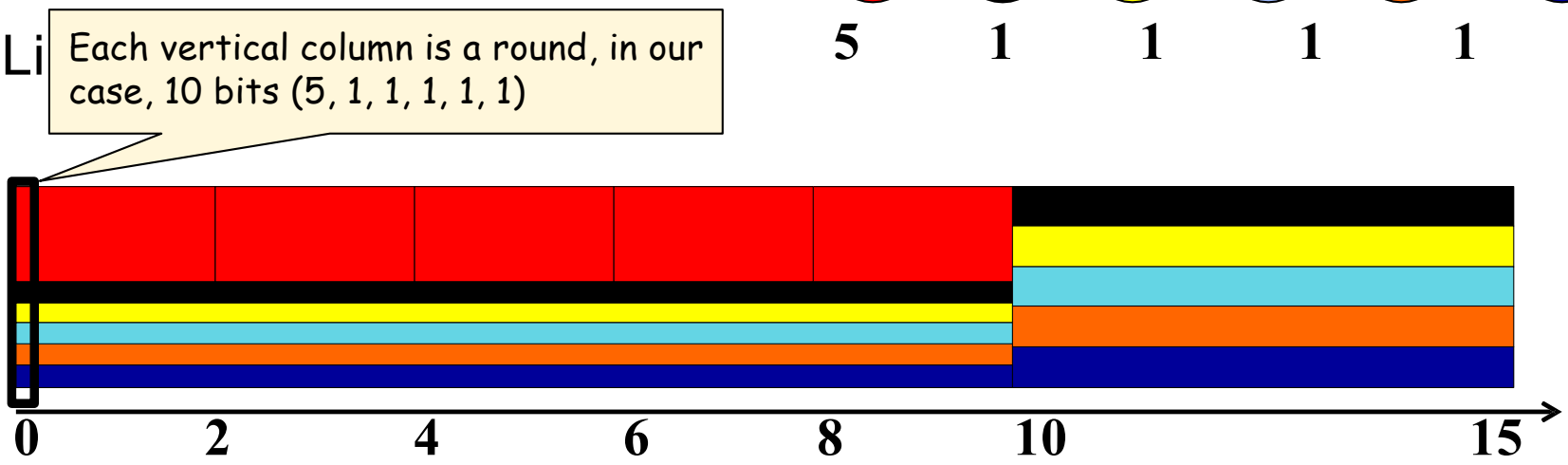




Generalized processor sharing example

- Red session has packets backlogged between 0 and 10
  - Other sessions have packets continuously backlogged
- Each packet has size 1
- Li



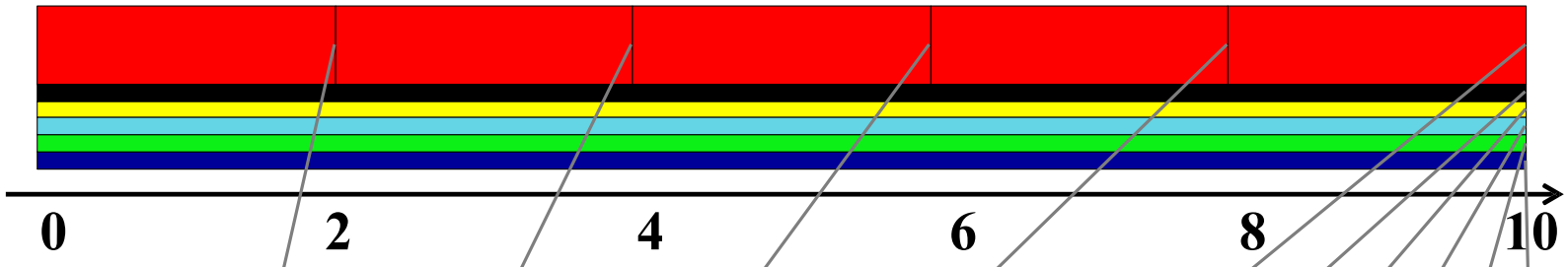


Packet approximation of GPS

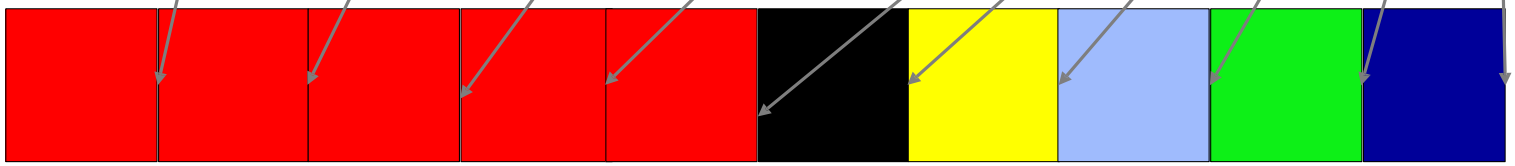
- Emulate GPS
- Select packet that finishes first in GPS assuming that there are no future arrivals

Approximating GPS with WFQ

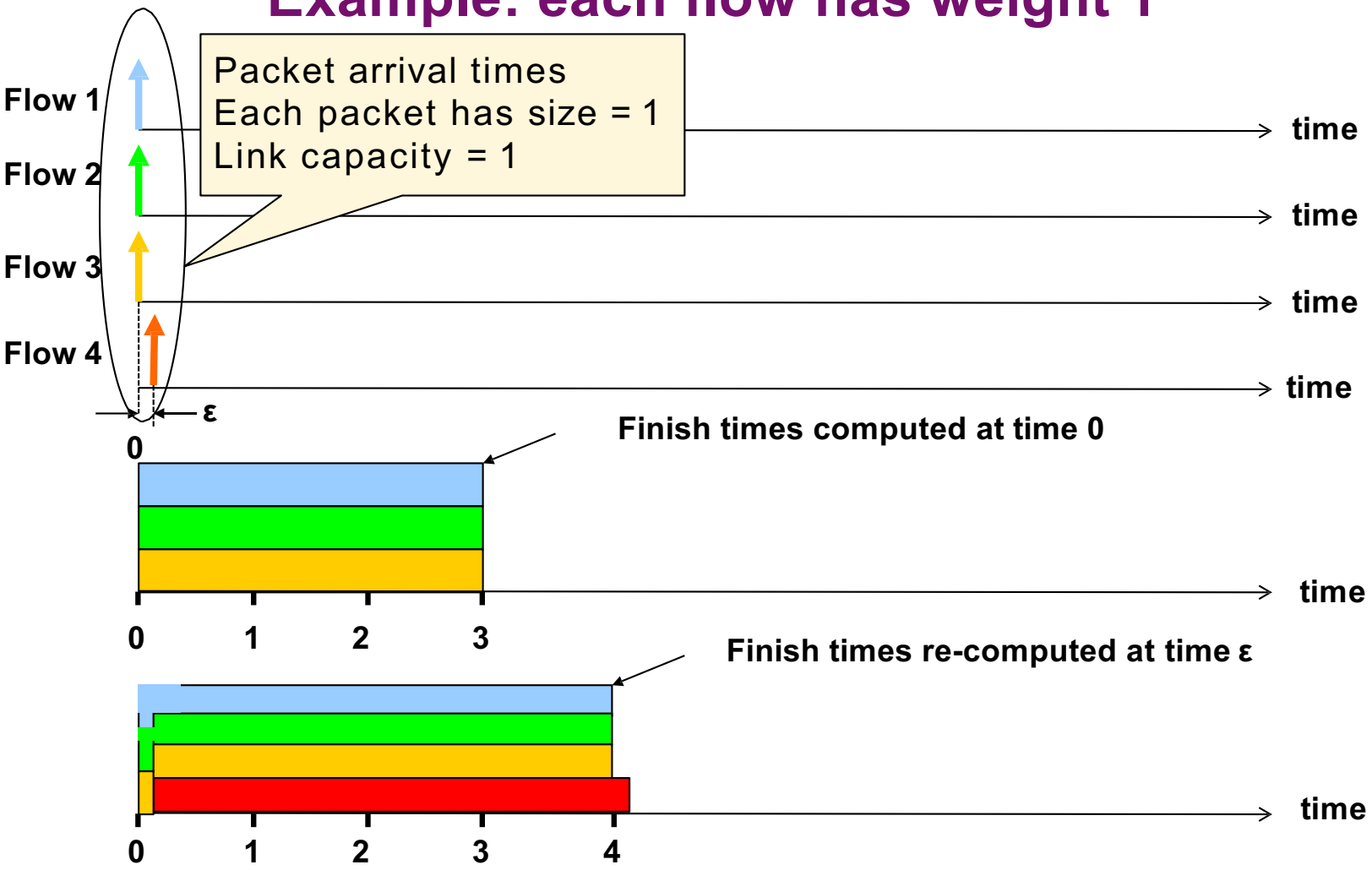
- Fluid GPS system service order



- Weighted Fair Queueing
  - Select the first packet that finishes in GPS



Example: each flow has weight 1

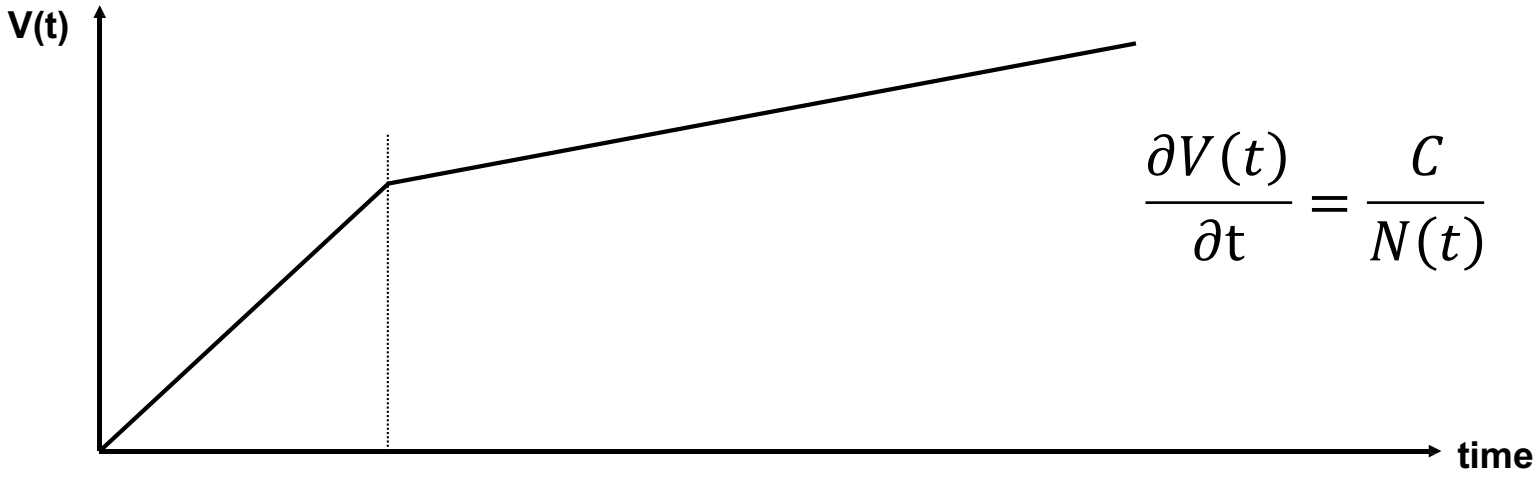


Solution: virtual time

- **Key observation**: while the finish times of packets may change when a new packet arrives…
  - The order in which packets finish doesn't!
  - Only the order is important for scheduling
- **Solution**: instead of the packet finish time maintain the number of rounds needed to send the remaining bits of the packet (virtual finishing time)
  - Virtual finishing time doesn't change when the packet arrives
- **System virtual time** – index of the round in the bit-by-bit round robin scheme

System virtual time: V(t)

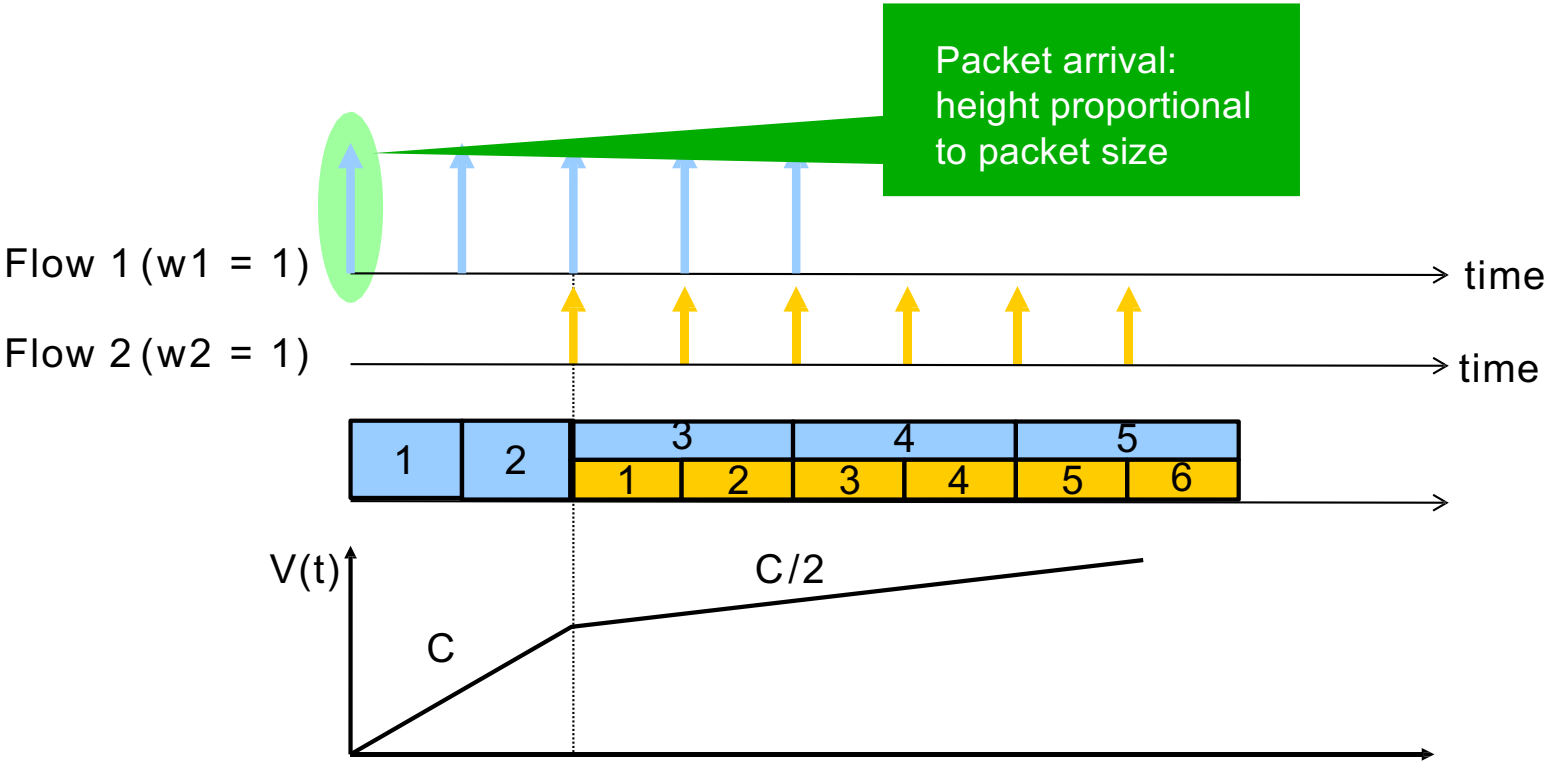
- Measure service, instead of time
- V(t) slope – normalized rate at which every backlogged flow receives service in the fluid flow system
  - C = link capacity
  - N(t) = total weight of backlogged flows in fluid flow system at time t



$$\frac{\partial V(t)}{\partial \mathrm{t}} = \frac{C}{N(t)}$$

System virtual time (V(t)): Example

- V(t) increases inversely proportionally to the sum of the weights of the backlogged flows



Fair queueing implementation

- Define
  - F_i^k virtual finishing time of packet k of flow i
  - a_i^k arrival time of packet k of flow i
  - L_i^k length of packet k of flow i
  - w_i weight of flow I
- The finishing time of packet $k+1$ of flow i is

$$F_i^{k+1} = \max(V(a_i^{k+1}), F_i^k) + L_i^{k+1} / w_i$$

Current round

Round when last packet of flow *i* finishes

of rounds it takes to serve new packet (i.e., packet *k+1* of flow *i*)

Round by which packet *k+1* is served

Early Eligible Virtual Deadline First (EEVDF)

EEVDF Scheduler

English

The "Earliest Eligible Virtual Deadline First" (EEVDF) was first introduced in a scientific publication in 1995 [1]. The Linux kernel began transitioning to EEVDF in version 6.6 (as a new option in 2024), moving away from the earlier Completely Fair Scheduler (CFS) in favor of a version of EEVDF proposed by Peter Zijlstra in 2023 [2-4]. More information regarding CFS can be found in CFS Scheduler.

https://www.kernel.org/doc/html/next/scheduler/sched-eevdf.html

A Proportional Share Resource Allocation Algorithm for Real-Time, Time-Shared Systems

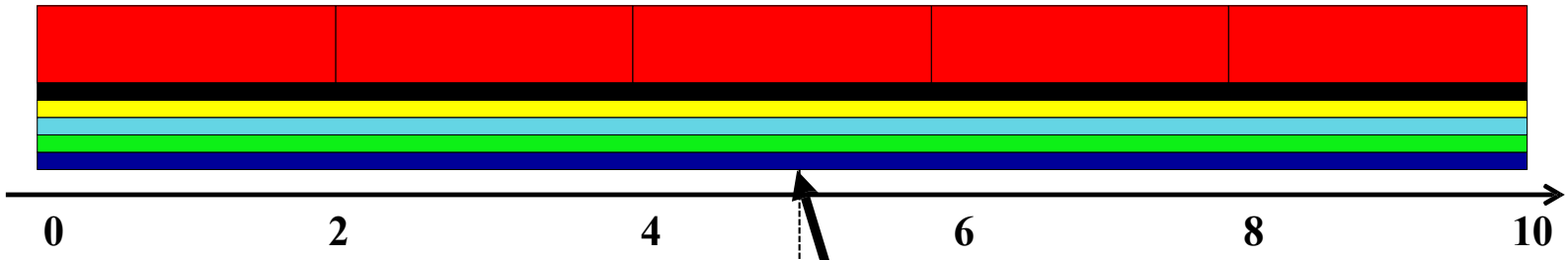
Ion Stoica * Hussein Abdel-Wahab † Kevin Jeffay‡ Sanjoy K. Baruah §
Johannes E. Gehrke ¶ C. Greg Plaxton ||

Abstract

We propose and analyze a proportional share resource allocation algorithm for realizing real-time performance in time-shared operating systems. Processes are assigned a weight which determines a share (percentage) of the resource they are to receive. The resource is then allocated in discrete-sized time quanta in such a manner that each process makes progress at a precise, uniform rate. Proportional share allocation algorithms are of interest because (1) they provide a time quantum. In addition, the algorithm provides support for dynamic operations, such as processes joining or leaving the competition, and for both fractional and non-uniform time quanta. As a proof of concept we have implemented a prototype of a CPU scheduler under FreeBSD. The experimental results shows that our implementation performs within the theoretical bounds and hence supports real-time execution in a general purpose operating system.

What problem does EEVDF try to solve?

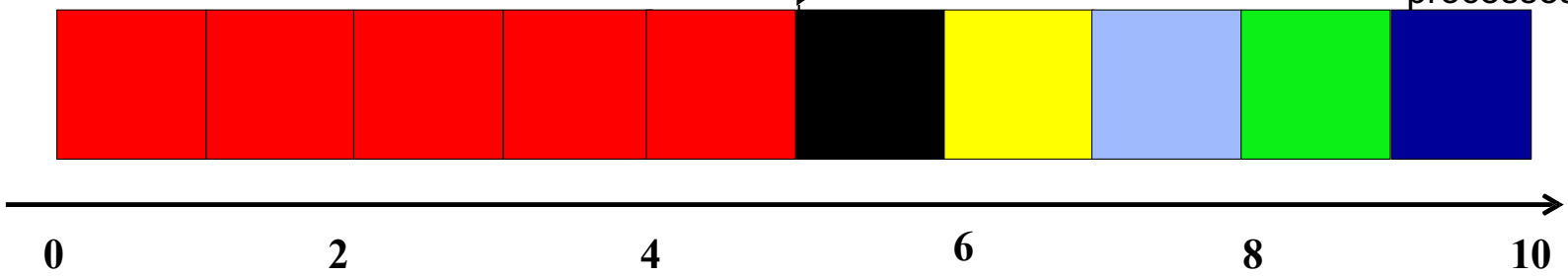
- Minimize lag: the difference between service received in real system vs fluid flow (idealized) system
  - Fluid system service order



Fluid system: red flow got 2.5
Real system: red flow got 5

Lag: -2.5 (worst case O(n) where n is the number of processes/flows)

  - Weighted Fair Queueing



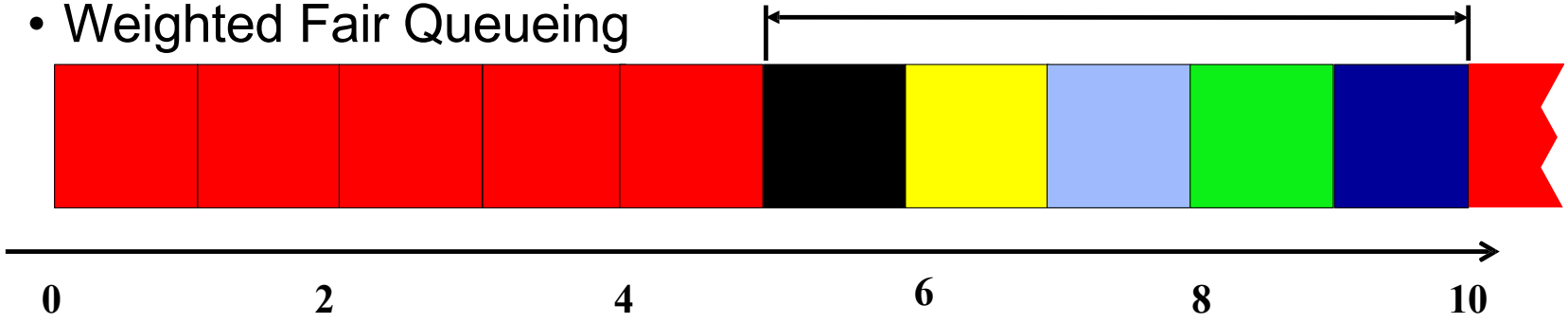
Why is this bad?

- Minimize lag: the difference between service received in real system vs fluid flow (idealized) system
  - Fluid system service order



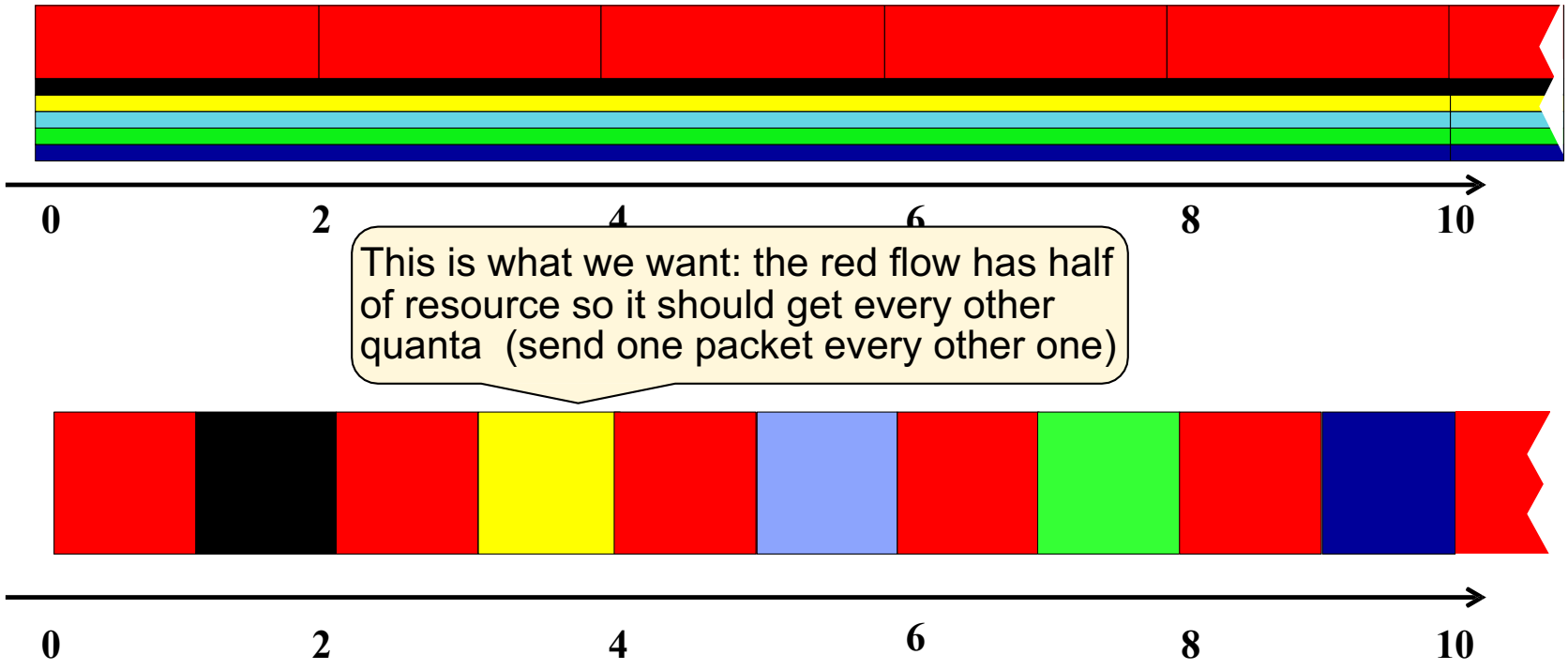
Red flow has 1/2 of resource capacity but can be denied service for n slots!

  - Weighted Fair Queueing



We want this!

- Minimize lag: the difference between service received in real system vs fluid flow (idealized) system
  - Fluid system service order

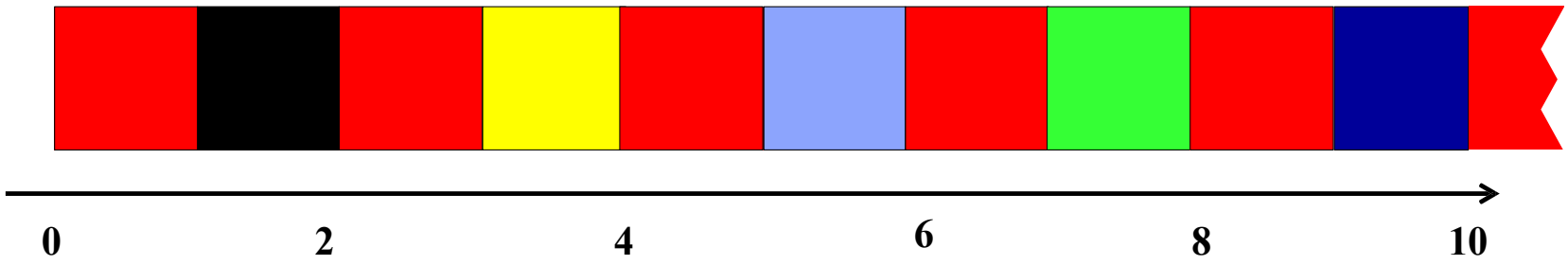


We want this!

- Minimize lag: the difference between service received in real system vs fluid flow (idealized) system
  - Fluid system service order



Lag <= 0.5 (independent on number of flows)!



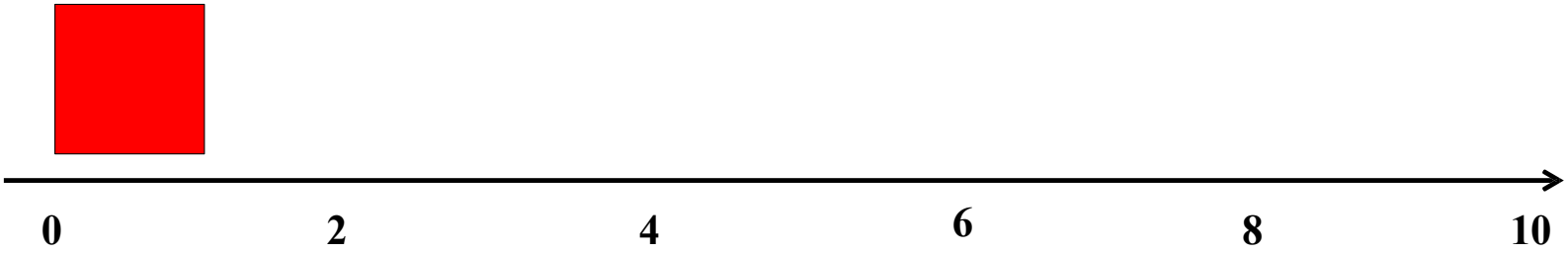
How?

- Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual starting time **smaller** than virtual time
  - Fluid system service order



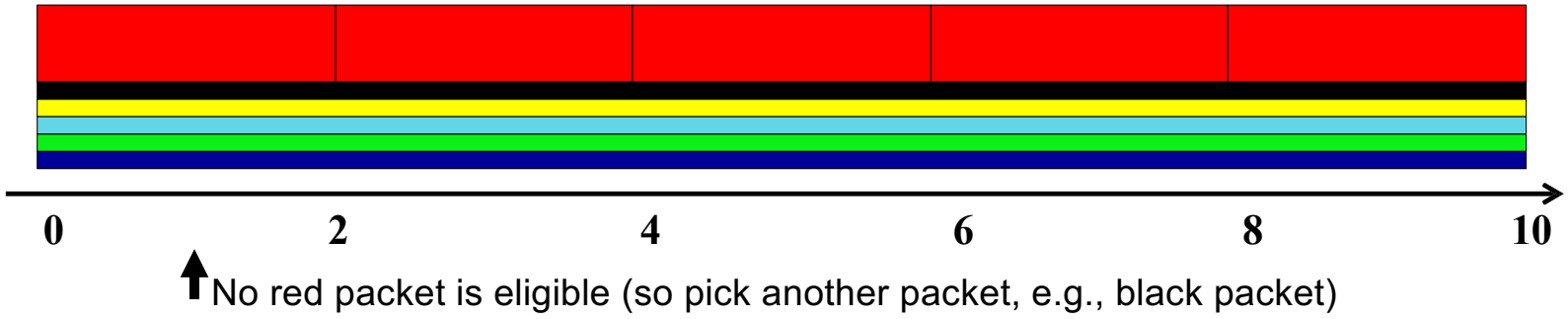
Only first of the red packets is eligible and has earliest deadline among all eligible packets so schedule it

  - Weighted Fair Queueing

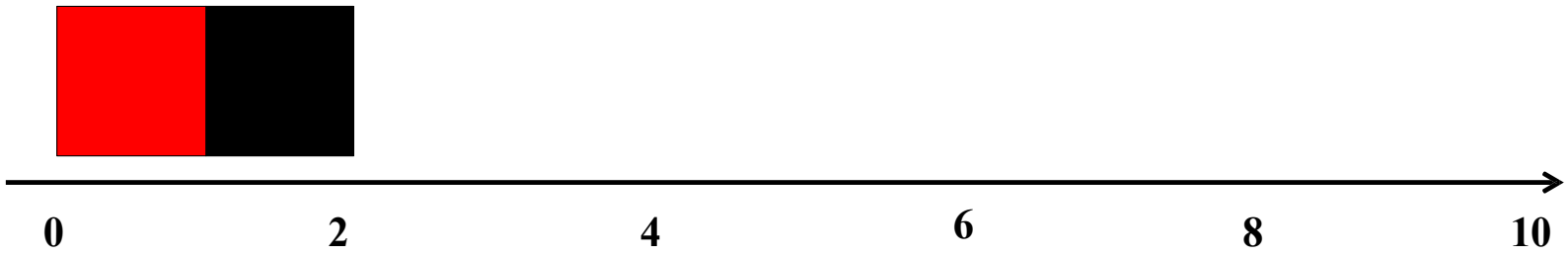


How?

- Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual starting time **smaller** than virtual time
  - Fluid system service order

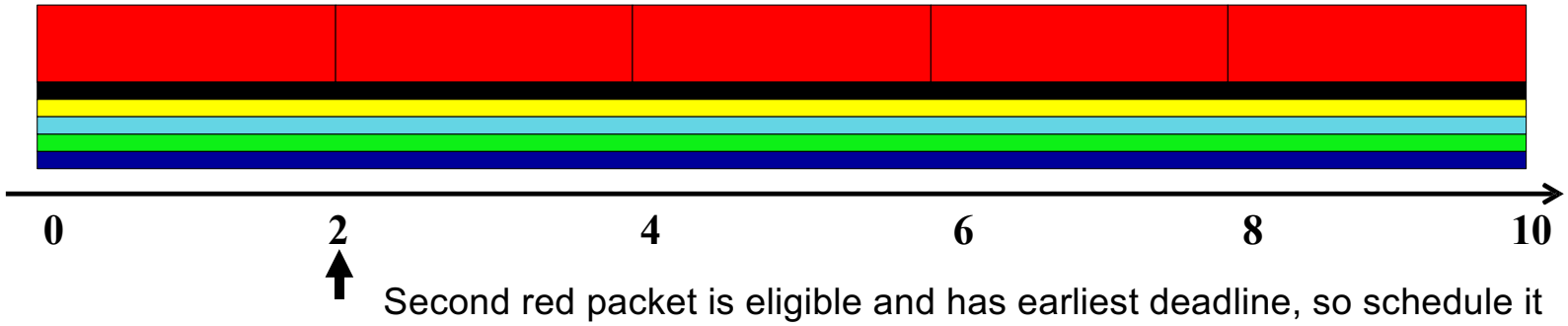


  - Weighted Fair Queueing

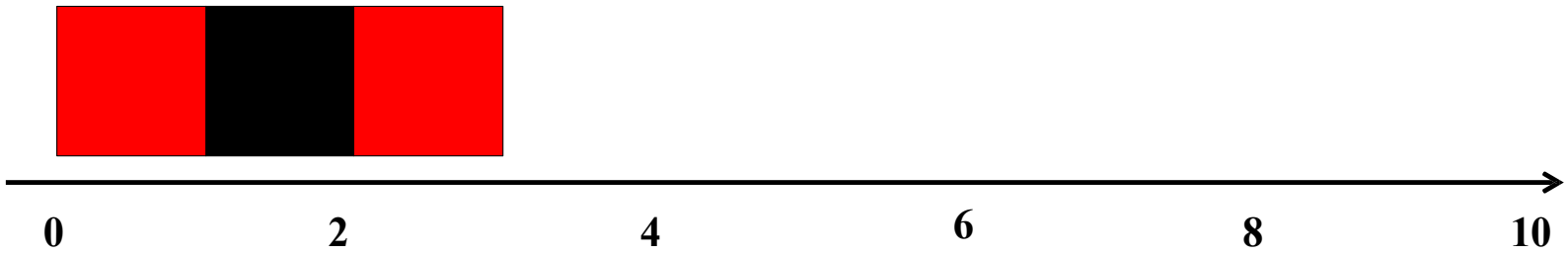


How?

- Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual starting time **smaller** than virtual time
  - Fluid system service order



  - Weighted Fair Queueing



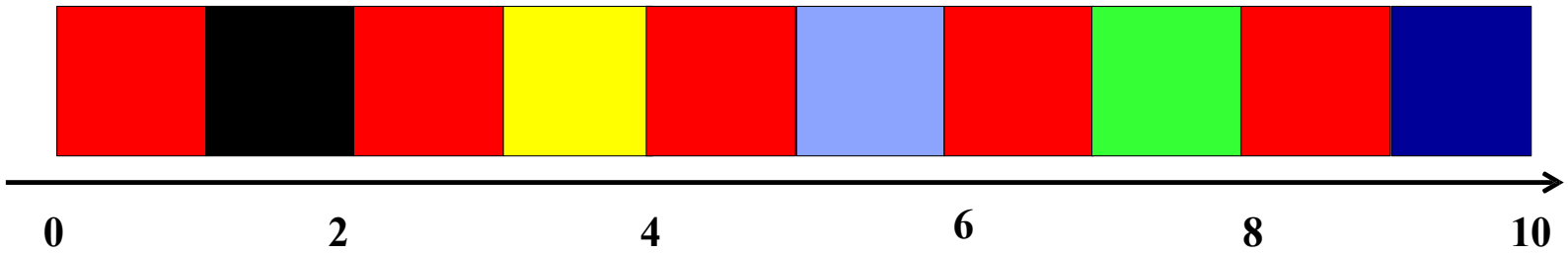
How?

- Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual starting time **smaller** than virtual time
  - Fluid system service order



Lag <= 0.5 (independent on number of flows)!

  - Weighted Fair Queueing



Comparison of proportional share disciplines

- n: number of flows / processes / users
- q: length of max time quanta

	Missing the deadline in fluid flow system	Service lag to fluid flow systems	Scheduling complexity	Comments
Fair sharing / queueing	q	O(n)*q	O(log(n))	- For networking q is max packet size - Data structure: priority queue ordered by finishing virtual time
EEVDF	q	O(1)*q	O(log(n))	More complex data structure
Lottery scheduling	O(\sqrt(n))*q	O(\sqrt(n))*q	O(1)	
Round robin	q	O(n)*q	O(n)	- If only one flow has packets to send, it takes O(n) to check every empty queue - Hard to handle diff. size time quanta

Conclusion

- Max-min fairness
  - Each job gets its “fair share”
  - Provides isolation: a job is guaranteed its fair share
- Weighted fair sharing:
  - A way to provide priorities: give higher weights to higher priority jobs
  - Avoid starvation: as long as a job has a non-negative weight it will get service
- Various implementations trading between complexity, accuracy, efficiency:
  - Fair sharing, Lottery scheduling, EEVDF

Reading for next lecture

- Operating System Concepts:
  - Chapter 5.5, 5.6, 8 (5.1-5.3 if you want to review today's content)

Thanks